

RadarIQOps

Colab Eğitim Odaklı Uygulama ve CI/CD Yol Haritası

72 planlı atomik commit

Model geliştirme, model testleri, tam eğitim ve değerlendirme Colab GPU ortamında; lokal çalışma yalnız entegrasyon ve servis doğrulamasında yürütülür. İlerleme test kanıtı, kalite kapısı ve geri alınabilir release üzerinden ölçülür.

Ana hüküm

RadioML/DeepSig kullanılıyorsa ürün modülasyon sınıflandırmasıdır. Gerçek radar hedef etiketli veri olmadan drone/kuş/araç iddiası yapılamaz. Model kodu, model testleri, kısa ve tam eğitim ile değerlendirme Colab GPU üzerinde yürütülür; lokal ortam yalnız immutable model artifact entegrasyonunu yapar.

KIRMIZI

Yapmadan geçme: mülakatta açıklayabilmeli, sıfırdan kurabilmeli ve hatasını debug edebilmelisin.

TURUNCU

Güçlü öğren: dokümantasyonla uygulayabilmeli, neden kullanıldığını ve trade-off'unu anlatabilmelisin.

1. Yönetici kararı

Bu proje için önerilen ana plan 72 atomik uygulama commit'idir. Entegrasyon sırasında gerçekten ortaya çıkan düzeltmeler ayrıca fix commit'i olabilir; doğal toplamın 72-84 aralığında kalması beklenir. Sayı uğruna commit bölmek veya bütün fazı tek commit'e sıkıştırmak doğru değildir.

Başlık	Karar
Plan tipi	Takvimsiz, Colab GPU eğitim + lokal entegrasyon ve kalite kapısı odaklı
Planlı commit	72 atomik uygulama commit'i
Gerçekçi toplam	72 planlı + yalnız ortaya çıkarsa 6-12 fix/integration commit'i; kabul bandı 72-84
Merge yaklaşımı	Kısa ömürlü branch + rebase merge; tüm fazı tek commit'e squash etme
İlk durdurucu kapı	Veri seti lisansı ve etiket semantiği; sonuç çıkmadan model/API sözleşmesi sabitlenmez
CI başlangıcı	Commit 012; lokal CI statik/API kontrollerini, Colab model kapısı ise tüm model testlerini yürütür
Üretim terfisi	Immutable image digest + immutable model version + manuel production onayı
Öğrenme vurgusu	21 kırmızı 'yapmadan geçme' commit'i + 27 turuncu güçlü destek commit'i

2. Hedef mimari

Veri + Lisans	DVC + Drive	Colab GPU	MLflow gate	Lokal FastAPI	OCI + K8s	Drift + Colab retrain
---------------	-------------	-----------	-------------	---------------	-----------	-----------------------

Model geliştirme, model testleri, eğitim ve değerlendirme Colab GPU üzerinde tamamlanır. Checkpoint ve raporlar Google Drive üzerinde kalıcılaştırılır; kabul edilen model checksum'lı immutable MLflow registry version olarak dışa aktarılır. Lokal ortam modeli eğitmez, yalnız sabit sürümü API/OCI/Kubernetes ile entegre eder. Production release manifesti image digest ile model version'ı bağlar.

Hedef repo yapısı

```
src/radariqops/{data, features, training, evaluation, tracking, inference, monitoring, retraining}
services/api/ # FastAPI app ve HTTP contract
configs/{data, features, training, colab, serving, drift}/
tests/{model_colab, integration, contract, smoke, fixtures}/
data/{raw, interim, processed, validation}/ # içerik DVC; Git'e büyük veri yok
notebooks/colab/ # veri, test, train, evaluate ve export girişleri
infra/{docker, k8s, monitoring}/
docs/{adr, runbooks, cards, architecture}/
.github/workflows/ # lokal integration CI, image, staging ve production
dvc.yaml params.yaml pyproject.toml Makefile README.md
```

MLOps öğrenme önceliği

KIRMIZI - Kesin öğren	TURUNCU - Güçlü öğren
Problem ve veri gerçeği - 004-005: görev/etiket semantiği ve doğru başarı ölçütü	Erken kalite ve güvenlik - 012-013: PR CI, dependency/license ve secret kontrolleri
Sürümlü veri ve leakage - 018, 021, 024-026: DVC, sızıntı önleme, train-fitted dönüşüm ve yeniden üretilebilir pipeline	Veri kalitesi - 002, 015, 017, 020, 022: veri kararı, schema, checksum, validation ve quarantine
Tekrarlanabilir eğitim - 030, 039, 041-042: training-serving eşitliği, determinizm ve tam run lineage	Model mühendisliği - 031, 033, 035, 037, 040, 043-045: baseline, config, checkpoint, test, MLflow ve değerlendirme
Model yönetimi - 046-047, 051: registry, promotion kapısı ve golden skew testi	Serving contract - 048-049, 052-054: ortak preprocessing, pinli model, health, validation ve API hata sözleşmesi
Immutable CI/CD - 059, 063-064: taranmış image, digest bazlı release, staging-production terfisi ve rollback	Çalışma zamanı altyapısı - 057, 061-062: non-root container, Kubernetes güvenliği ve kind smoke
Üretim geri-besleme döngüsü - 065, 069-071: metrik, drift, audit'li retraining, onay ve rollback	Operasyon ve anlatılabilirlik - 066-068, 072: alarm, dashboard, mahremiyetli drift girdisi ve runbook

3. Commit ve branch kuralları

- Bir commit tek bir gözlenebilir davranış değiştirir; ilgili test ve küçük doküman aynı commit'tedir.
- Sırf commit sayısını doldurmak için dosya başına commit atılmaz. WIP/fixup commit'leri ana dala girmeden düzenlenir.
- Tüm fazı tek squash commit'e çevirmek yasaktır; 72 maddelik izlenebilir geçmiş korunur.
- Her commit en az fast CI'ı geçer. Path bazlı ağır işler yalnız ilgili alan değişince koşar.
- Kırmızı main kabul edilmez. Acil düzeltme ayrı fix commit'i ve regression testiyle gelir.
- Generated data, model binary, mlruns, secret ve büyük örnekler Git'e alınmaz.
- Model promotion ile uygulama deployment'ı iki ayrı onay hattıdır; release manifesti ikisini bağlar.

4. CI/CD iş akışları

Hat	Tetik	İşler	Kapı
PR-fast	Her PR	Lock, lint, format, typecheck, model-dışı unit/integration, secret/license/dependency	Model testleri burada koşmaz; Colab kanıtı ayrı kapıdır
PR-data	Data kodu/config değişince	Fixture download, validate, preprocess, split, dvc repro	Gerçek büyük veri indirilmez
Colab-model	Training/model değişince manuel dispatch	Feature/model testleri, one-batch overfit, kısa GPU run, lineage ve artifact	Model kapısı Colab kanıtı olmadan geçmez
Local-integration	API/inference değişince	Export bundle doğrulama, golden parity, OpenAPI, HTTP ve latency smoke	Lokal ortam eğitim yapmaz; skew merge'i engeller
PR-image-k8s	Container/infra değişince	Image build, SBOM/scan, non-root smoke, manifest policy, kind	PR image'ı push edilmez
Main-release	Main'e merge	Tüm kapılar, OCI push, SHA/digest/provenance	Tag overwrite yok
Staging-CD	Yeni digest	Aynı digest deploy, smoke, sentetik trafik, dashboard kontrolü	Başarısızlık production'ı engeller
Production-CD	SemVer + onay	Aynı digest ve pinli model version; post-deploy health	Önceki digest/model rollback girdisi
Colab-full-training	Manuel Colab GPU	Gerçek DVC veri, tüm model testleri, train, evaluate, immutable candidate export	Notebook run kanıtı ve production terfisi ayrı kapı

Zorunlu soy ağacı

Katman	Kaydedilecek kimlik
Kod	Git SHA, dirty flag, release tag
Veri	Kaynak manifesti, checksum, DVC revision, split hash
Config	Çözümlemiş config ve config hash
Model	MLflow run ID, registry model/version, preprocessing schema
Container	OCI digest, SBOM, scan ve provenance
Deployment	Ortam, image digest, model version, onaylayan, zaman, rollback hedefi

5. Teknik kabul matrisi

Alan	Kabul ölçütü
Veri	Lisans/atıf kayıtlı; schema hataları raporlu; duplicate/group leakage yok; aynı input aynı manifest/hash.
Model	Baseline ile aynı split; macro-F1 ve kritik recall öncelikli; SNR dilimleri ve güven aralıkları raporlu.
Promotion	Challenger tanımlı kalite kapılarını geçer. Etiket yoksa drift yalnız candidate/investigation tetikler; otomatik production yok.
API	Contract sürümlü; bozuk/aşırı payload güvenli 4xx; model/preprocess sürümü ve request_id görünür; raw IQ loglanmaz.
Performans	Hedef donanım belirtilir; warmup sonrası p95/p99, throughput, model size ve memory ölçülür; eşik baseline sonrası sabitlenir.
Container/K8s	Non-root, pinned base, SBOM/scan, probe, request/limit, rollout ve digest bazlı deploy.
Observability	Gerçek örnek trafik dashboard'u doldurur; alarmlar sentetik ihlalle çalışır ve runbook'a gider.
Drift	Referans/current version'lı; min sample, ardışık pencere ve cooldown var; sentetik shift pozitif, normal fixture negatif.
Rollback	Önceki image digest ve model version bulunabilir; smoke testle geri dönüş kanıtlanır.

6. Faz özeti

Faz	Commit	Amaç	Çıkış kapısı
Faz 0 - Ürün gerçeği ve veri kararı	001-006	Yanlış problem üzerinde doğru görünen bir sistem kurulmasını engellemek.	G0: Veri lisansı, etiket anlamı, görev adı ve kabul metrikleri yazılı olarak onaylı. Karar çıkmazsa uygulama durur.
Faz 1 - Repo, kalite sözleşmesi ve erken CI	007-014	İlk üretim kodundan önce aynı kuralların yerelde ve CI'da uygulanmasını sağlamak.	G1: Temiz lokal klonda kurulum, statik kontroller, model-dışı unit/integration ve güvenlik kontrolleri yeşil; model testleri Colab kapısındadır.
Faz 2 - Sürümlü ve doğrulanmış veri hattı	015-026	Aynı ham veriden aynı işlenmiş veri ve bölme indekslerini yeniden üretebilmek.	G2: Fixture üzerinde dvc repro temiz çalışıyor; gerçek veri manifesti, doğrulama raporu ve değişmez split indeksleri mevcut.
Faz 3 - Colab feature, model testleri ve GPU eğitim çekirdeği	027-039	Tüm feature/model testlerini ve eğitimi Colab GPU üzerinde deterministik, yeniden başlatılabilir ve karşılaştırılabilir hale getirmek.	G3: Temiz Colab GPU runtime'ında tüm model testleri, baseline ve 1D CNN aynı split'te koşuyor; one-batch overfit, resume ve tekrarlanabilirlik kanıtları Drive'a yazılıyor.
Faz 4 - MLflow, değerlendirme ve model yönetişimi	040-047	Bir modelin hangi kod, veri ve config ile üretildiğini ve neden terfi ettiğini kanıtlamak.	G4: Registry'de imzalı input örneği olan candidate var; baseline/CNN ve SNR analizi aynı test politikasında raporlu.
Faz 5 - Lokal model entegrasyonu ve API	048-056	Colab'den çıkan immutable model paketini lokal ortamda yeniden eğitim yapmadan güvenli ve gözlenebilir bir HTTP servisine entegre etmek.	G5: Model sürümü görünür; tekli/batch API contract, hata davranışı, golden sample ve latency smoke testleri geçiyor.
Faz 6 - Container, Kubernetes ve gerçek CI/CD	057-064	Aynı immutable uygulama image'ını staging ve production'a kanıt zinciriyle taşımak.	G6: OCI digest üretilmiş, taranmış, kind/staging smoke geçmiş; production terfisi aynı digest ve manuel onayla yapılabilir.
Faz 7 - Gözlemlenebilirlik, drift, retraining ve ürünleşme	065-072	Sistemin bozulmasını görmek, kanıtsız otomasyonu engellemek ve güvenli model iyileştirme döngüsünü kapatmak.	G7: Trafik-dashboard-alert-drift-candidate-promotion/ret-ro llback zinciri uçtan uca gösterilmiş; v1.0.0-mvp yayımlanabilir.

7. 72 commit'lik ayrıntılı yol haritası

Her satır ana dala girecek tek mantıksal değişikliği, uygulanacak kapsamı ve merge için gereken kanıtı tanımlar.

Faz 0 - Ürün gerçeği ve veri kararı (001-006)

Yanlış problem üzerinde doğru görünen bir sistem kurulmasını engellemek.

G0: Veri lisansı, etiket anlamı, görev adı ve kabul metrikleri yazılı olarak onaylı. Karar çıkmazsa uygulama durur.

No	Önerilen commit	Uygulama kapsamı	CI / kabul kanıtı
001	docs(product): define product charter and non-goals	docs/product-charter.md içinde kullanıcı, karar çıktısı, online/offline kullanım, veri girdisi ve kapsam dışı öğeler tanımlanır. Gerçek radar hedef verisi yoksa ürün 'hedef sınıflandırıcı' diye sunulmaz.	Doküman incelemesi; tüm hedefler ölçülebilir, kapsam dışı maddeler açık ve çelişkisiz olmalı.
002	docs(data): add dataset decision matrix	RadioML/DeepSig ile uygun radar hedef veri setleri lisans, erişim, sınıf anlamı, SNR, örnek şekli, boyut ve yeniden dağıtım hakkı bakımından puanlanır.	Karar matrisi her aday için kaynak, lisans kanıtı ve red nedeni içerir; belirsiz lisanslı veri elenir.
003	spike(data): add dataset inspection CLI	radariq data inspect komutu seçili adaydan küçük bir örnek açar; shape, dtype, label, SNR, grup/sequence kimliği ve temel istatistikleri JSON olarak üretir.	Lisans açısından güvenli fixture üzerinde CLI smoke testi; beklenen şema alanları snapshot ile doğrulanır.
004	docs(adr): record dataset and task decision	ADR-001 görev adını sabitler: gerçek radar hedef sınıflandırması veya modülasyon sınıflandırması. Etiket semantiği, bilinen sınırlamalar ve yeniden değerlendirme koşulları kaydedilir.	G0 karar kontrol listesi; 'drone/kuş/araç' iddiası yalnız gerçek etiket kanıtıyla geçer.
005	docs(metrics): define measurable acceptance policy	Birincil macro-F1, sınıf recall, SNR dilimi performansı, kalibrasyon, model boyutu, p95 gecikme ve hata oranı tanımlanır. Sabit yüzde 90 iddiası kaldırılır.	Her metrik için veri bölümü, hesap yöntemi, yönü, ilk eşik belirleme zamanı ve sorumlu artifact tanımlı olmalı.
006	docs(architecture): add system context and risk register	Veri-DVC-eğitim-MLflow-API-container-Kubernetes-monitoring-drift-retraining zinciri, güven sınırları ve kod/veri/model soy ağacı çizilir. Risk sahibi ve azaltma ölçütü eklenir.	Mimari inceleme; her çalışma zamanı bileşeni, artifact kaynağı ve geri alma noktası gösterilir.

Faz 1 - Repo, kalite sözleşmesi ve erken CI (007-014)

İlk üretim kodundan önce aynı kuralların yerelde ve CI'da uygulanmasını sağlamak.

G1: Temiz lokal klonda kurulum, statik kontroller, model-dışı unit/integration ve güvenlik kontrolleri yeşil; model testleri Colab kapısındadır.

No	Önerilen commit	Uygulama kapsamı	CI / kabul kanıtı
007	chore(repo): scaffold production package layout	src/radariqops altına ana paketler; services/api, tests, configs, infra ve docs dizinleri açılır. notebooks/colab altında 00_bootstrap, 10_model_tests, 20_train ve 30_evaluate_export giriş notebook'ları oluşturulur.	Paket import smoke testi ve repo ağacı kontrolü; notebook yalnız orkestrasyon girişidir, model mantığı src paketlerinde kalır ve lokal ortam notebook çalıştırmaz.
008	build(python): pin runtime and dependency groups	Python sürümü ve bağımlılıklar sabitlenir. Lokal core/serve/dev profili model eğitmez; Colab için GPU uyumlu train lock dosyası ve kurulum hücresi ayrı tutulur.	Temiz lokal serve kurulumu ve temiz Colab GPU runtime kurulumu; iki ortamda ortak inference import testi.
009	build(dev): add repeatable developer commands	Makefile veya eşdeğer task runner ile lokal setup, lint, typecheck, integration-test, api-smoke ve compose-up komutları; Colab notebook girişleri için data, model-test, train, evaluate ve export komutları eklenir.	CI aynı komutları çağırır; README quickstart komutları kopyala-çalıştır testinden geçer.
010	style(repo): configure lint format and pre-commit	Ruff format/lint, import sırası, YAML/Markdown kontrolleri ve secret tarama hook'ları yapılandırılır. Otomatik düzeltilebilir kurallar yerelde uygulanır.	pre-commit run --all-files temiz; CI yalnız kontrol modunda çalışır.
011	test(core): establish pytest strategy and markers	Model testleri Colab marker/grubuna; API, contract, artifact yükleme ve smoke testleri lokal integration grubuna ayrılır. Deterministik seed fixture'ları ve kanıt manifesti eklenir.	Lokal test komutu model testlerini toplamaz; Colab test komutu feature, gradient, overfit, reproducibility ve evaluation testlerinin tamamını toplar.
012	ci(pr): add fast quality workflow	Pull request için lock doğrulama, lint, format, typecheck ve lokal integration testleri eklenir. Model kodu değişirse Colab run manifesti ve notebook kanıtı zorunlu kontrol olur.	Bilinçli lint ve test hataları workflow'u durdurur; gerekli işler branch protection listesine yazılır.
013	ci(security): add secret dependency and license checks	Secret taraması, bağımlılık zafiyet taraması, izin verilen lisans politikası ve workflow permission minimizasyonu eklenir.	Yüksek/kritik bulgu ve yasak lisans merge'i engeller; istisna süreli ve kayıtlı olmalıdır.
014	ci(policy): define commit PR and release rules	Conventional Commits, PR şablonu, CODEOWNERS, değişiklik notu ve rebase merge politikası tanımlanır. WIP/fixup commit'leri merge öncesi düzenlenir.	Commit mesajı kontrolü ve PR kontrol listesi geçmeden merge yok; ana dal korumalıdır.

Faz 2 - Sürümlü ve doğrulanmış veri hattı (015-026)

Aynı ham veriden aynı işlenmiş veri ve bölme indekslerini yeniden üretebilmek.

G2: Fixture üzerinde dvc repro temiz çalışıyor; gerçek veri manifesti, doğrulama raporu ve değişmez split indeksleri mevcut.

No	Önerilen commit	Uygulama kapsamı	CI / kabul kanıtı
015	feat(data): define IQ schema and data contracts	IQ gösterimi [N,2,L] veya complex64, label, SNR, group_id, sample_id ve kaynak sürümü için tipli sözleşme oluşturulur; schema_version alanı eklenir.	Geçerli ve hatalı örneklerle sözleşme testleri; geriye uyumsuz şema değişikliği açık sürüm artışı gerektirir.
016	feat(data): implement configurable acquisition adapters	HTTP, yerel dosya veya kullanıcı tarafından sağlanan arşiv için kaynak adaptörü yazılır. İndirme yarda kalırsa atomik tamamlanma ve tekrar deneme uygulanır.	Fixture sunucusu/yerel arşiv ile idempotency testi; ham dosya ikinci koşuda değişmemeli.
017	feat(data): verify checksums licenses and manifests	Her kaynak için SHA-256, lisans kimliği, atıf, indirme zamanı, dosya boyutu ve erişim yöntemi data manifestine yazılır.	Checksum uyumsuzluğu pipeline'ı durdurur; eksik lisans metadata'sı G2'yi geçemez.
018	build(dvc): initialize data and artifact versioning	DVC başlatılır; data/raw ve büyük artifact'lar Git dışında tutulur. Yerel geliştirme ve CI/full-run için remote profilleri belgelenir.	Git'te büyük dosya kontrolü; dvc status ve fixture pull/push smoke testi.
019	feat(data): add immutable raw ingestion stage	İndirilen arşiv açılır, örnek kimlikleri deterministik üretilir ve raw stage çıktısı yalnız yeni kaynak sürümüyle değişir.	Aynı input iki koşuda aynı manifest/hash verir; raw alanına yerinde mutation testi başarısız olur.
020	feat(data): validate shape values and labels	Shape/uzunluk, dtype, boş kayıt, NaN/Inf, sabit sinyal, amplitude/power sınırı, label kümesi ve SNR aralığı kontrolleri uygulanır.	Her ihlal için ayrı unit test; raporda hata kodu, örnek kimliği ve sayım bulunur.
021	feat(data): detect duplicates groups and leakage	Exact/near duplicate, aynı sequence veya kaynak grubunun farklı split'lere sızma riski belirlenir. group_id üretim kuralı veri setine özgü adaptörde tutulur.	Sentetik sızıntı fixture'ı yakalanır; karar verilmemiş group_id ile split üretilemez.
022	feat(data): quarantine invalid samples with report	Bozuk kayıtlar raw'dan silinmez; quarantine manifestine alınır. JSON ve insan okunur HTML/Markdown doğrulama raporu üretilir.	Toplam = kabul + karantina eşitliği, hata dağılımı ve örneklenmiş kanıt CI artifact'ı olarak yayımlanır.
023	feat(data): generate reproducible EDA artifacts	Sınıf/SNR dağılımı, uzunluk, I/Q mean-std, power ve örnek spektrumlar notebook bağımsız CLI ile üretilir.	Sabit fixture için sayısal özet snapshot'ı; grafiklerin veri kaynağı ve run metadata'sı kayıtlıdır.
024	feat(data): add train-fitted preprocessing	DC offset giderme ve amplitude/power normalizasyonu config kontrollü uygulanır. Fit edilen istatistikler artifact olur; validation/test kendi istatistiğini kullanmaz.	NaN, zero-power ve aşırı amplitude testleri; inverse/kararlılık ve train-only fit kanıtı.
025	feat(data): create group-aware deterministic splits	Stratified group-aware train/validation/test indeksleri seed ile üretilir, test seti kilitlenir ve split manifestine sınıf/SNR dengesi yazılır.	Gruplar kesişmez, aynı seed aynı indeksleri verir; test split'i model seçim kodundan erişilemez.
026	build(dvc): wire validation preprocessing and split pipeline	dvc.yaml ve params.yaml raw -> validate -> preprocess -> split -> report zincirini bağlar. Gerçek veri ve veri/model testleri Colab'de; lokal ortam yalnız indirilen artifact manifestini doğrular.	Temiz Colab runtime'ında DVC pull/repro kanıtı ve aynı split hash'i üretilir; lokal ortam model verisini işlemeyen export manifestini doğrular.

Faz 3 - Colab feature, model testleri ve GPU eğitim çekirdeği (027-039)

Tüm feature/model testlerini ve eğitimi Colab GPU üzerinde deterministik, yeniden başlatılabilir ve karşılaştırılabilir hale getirmek.

G3: Temiz Colab GPU runtime'ında tüm model testleri, baseline ve 1D CNN aynı split'te koşuyor; one-batch overfit, resume ve tekrarlanabilirlik kanıtları Drive'a yazılıyor.

No	Önerilen commit	Uygulama kapsamı	CI / kabul kanıtı
027	feat(features): add canonical IQ tensor representation	İşlenmiş sinyal PyTorch için sabit channel-first tensor'a çevrilir; padding/cropping politikası config ile ve mask metadata'sıyla uygulanır.	Girdi/çıkış shape, dtype, sıra ve deterministik padding testleri.
028	feat(features): add amplitude phase representation	Amplitude ve sarılmış/faz dönüşümü modüler transform olarak eklenir; phase belirsizlikleri ve zero-amplitude davranışı tanımlanır.	Bilinen kompleks sinyallerde sayısal doğruluk ve NaN üretmeme testleri.
029	feat(features): add optional spectral representations	FFT ve gerekliyse spectrogram yalnız config ile seçilen alternatif feature olur; pencere, overlap ve ölçek parametreleri sürümlenir.	Saf ton fixture'ında beklenen frekans tepe noktası; feature artifact hash'i config değişimine duyarlı.
030	test(features): add leakage and golden transform suite	Ortak preprocessing paketinin eğitim ve serving kullanımını kanıtlayan golden vektörler ile train-only istatistik testleri eklenir.	Aynı sample train ve inference yolunda tolerans içinde aynı tensörü üretir.
031	feat(baseline): implement classical baseline runner	Logistic regression veya küçük MLP, sabit split ve aynı feature'larla çalışır; macro-F1, sınıf recall ve confusion matrix üretir.	Fixture üzerinde hızlı integration testi; baseline artifact'ları ve seed yeniden üretilebilir.
032	feat(training): add deterministic dataset and dataloader	Dataset/DataLoader, worker seed, sampler, batch contract ve CPU pinning davranışı tanımlanır; sample_id batch boyunca korunur.	0/1/çok worker sıralama ve seed testleri; batch şeması contract testi.
033	feat(training): add typed training configuration CLI	Model, feature, optimizer, lr, batch, epoch, scheduler, seed, cihaz ve output ayarları doğrulanan config şemasına ve train CLI'na bağlanır.	Hatalı config erken ve anlaşılır hata verir; çözümlenmiş config her run'a kopyalanır.
034	feat(training): implement Colab GPU training loop	Colab GPU train/validation döngüsü, gradient yönetimi, mixed precision, metrik toplama ve cihaz doğrulaması eklenir. Eğitim GPU yoksa başlamaz; lokal CPU eğitim yolu kapsam dışıdır.	Temiz Colab GPU runtime'ında küçük fixture bir epoch biter; GPU bağlı değilse eğitim açık hatayla durur ve CPU fallback yapılmaz.
035	feat(training): add checkpoints early stop and resume	Latest/best checkpoint, optimizer/scheduler state, epoch ve RNG state her epoch Google Drive'a atomik kaydedilir. Colab oturum kopmasına karşı resume zorunludur.	Kesintili run'ın resume sonucu kesintisiz run ile tolerans içinde eşleşir; bozuk checkpoint güvenli hata verir.
036	feat(model): implement configurable 1d cnn	Conv1D, normalization, activation, pooling, dropout ve classifier katmanları config tabanlı kurulur; parametre sayısı raporlanır.	Farklı input length/batch için forward/backward shape testleri ve serialization testi.
037	test(model): add one-batch overfit and gradient checks	Küçük bir batch üzerinde kaybın anlamlı düşmesi, tüm beklenen parametrelerde gradient ve finite output doğrulanır.	Overfit ve gradient testleri Colab model-test hücresinde çalışır; başarılı test raporu ve runtime kimliği olmadan model kapısı geçmez.
038	feat(training): add evidence-based imbalance handling	Class weight veya weighted sampler config seçenekleri eklenir; seçim EDA dağılımına dayanır ve aynı anda iki yöntem varsayılan kullanılamaz.	Sampler dağılım testi, weight hesap doğruluğu ve azınlık sınıfı içeren fixture regression testi.
039	test(training): add reproducibility and performance baselines	Aynı seed/config/data revision ile iki kısa run karşılaştırılır; eğitim süresi, örnek/saniye ve peak memory başlangıç baseline'i kaydedilir.	İki temiz Colab GPU run metrikleri tolerans içinde eşleşir; süre, VRAM ve throughput raporu Drive artifact'ıdır. Lokal makinede hiçbir model testi veya eğitim çalıştırılmaz.

Faz 4 - MLflow, değerlendirme ve model yönetiřimi (040-047)

Bir modelin hangi kod, veri ve config ile üretildiğini ve neden terfi ettiğini kanıtlamak.

G4: Registry'de imzalı input örneğı olan candidate var; baseline/CNN ve SNR analizi aynı test politikasında raporlu.

No	Önerilen commit	Uygulama kapsamı	CI / kabul kanıtı
040	feat(tracking): configure MLflow experiments and backend	Colab'den erişilebilen kalıcı MLflow tracking URI ve artifact store yapılandırılır. Kimlik bilgileri Colab Secrets'tan alınır; notebook veya Git'e yazılmaz.	Temiz Colab runtime'ında run aç/kapat ve artifact upload testi; oturum kopması veya upload hatası doğru status ile kaydedilir.
041	feat(tracking): log full run lineage	Git SHA, DVC/data revision, split/config hash, seed, Colab runtime kimliği, Python/bağımlılık, GPU modeli ve CUDA bilgisi her run'a yazılır.	Eksik lineage alanı training integration testini başarısız yapar; run'dan kaynak girdiler geri bulunabilir.
042	feat(tracking): log metrics models and artifacts	Epoch metrikleri, checkpoint, test raporları, confusion matrix, classification report, config ve log özeti Colab'den kalıcı MLflow/Drive artifact alanına yüklenir.	Artifact varlığı ve isim sözleşmesi test edilir; hassas yol/token log'a yazılmaz.
043	feat(evaluation): add locked test and SNR analysis	Yalnız seçilmiş candidate için kilitli test değerlendirmesi; genel, sınıf bazlı ve SNR dilimli metrikler ile bootstrap güven aralığı üretilir.	Model seçimi test metriğine erişemez; rapor sample count ve dengesiz sınıf uyarılarını içerir.
044	perf(evaluation): benchmark latency size and throughput	Colab GPU'da eğitim throughput, epoch süresi, peak VRAM ve model boyutu ölçülür. Gerçek servis p50-p95-p99 ölçümü lokal integration fazında yapılır.	Donanım ve run koşulları raporda; smoke bütçesini aşan bariz regresyon CI'da uyarı/engel politikasıyla yakalanır.
045	feat(evaluation): compare baseline and cnn fairly	Aynı split, feature ve metrik hesaplayıcıyla baseline-CNN farkı; güven aralığı, sınıf/SNR trade-off ve maliyet özeti tek raporda gösterilir.	Karşılaştırılan run'ların data/split hash'i eşit değilse rapor üretimi durur.
046	feat(registry): register immutable candidate model	Colab'de geçen model signature, input example, label map, preprocessing contract ve golden outputs ile checksum'lı export bundle oluşturulur; immutable registry version kaydedilir.	Kaydedilen model yeni süreçte yüklenip golden sample tahmini verir; schema_version uyumluluğı kontrol edilir.
047	feat(governance): enforce promotion policy and model card	Macro-F1/recall/latency/size kapıları, non-inferiority veya anlamlı iyileşme kuralı, model card ve ret nedenleri kodlanır. Test etiketi yoksa otomatik production terfisi yasaktır.	İyi ve kötü challenger fixture'ları; başarısız aday champion olamaz. v0.2.0-model G4 ile etiketlenebilir.

Faz 5 - Lokal model entegrasyonu ve API (048-056)

Colab'den çıkan immutable model paketini lokal ortamda yeniden eğitim yapmadan güvenli ve gözlenebilir bir HTTP servisine entegre etmek.

G5: Model sürümü görünür; tekli/batch API contract, hata davranışı, golden sample ve latency smoke testleri geçiyor.

No	Önerilen commit	Uygulama kapsamı	CI / kabul kanıtı
048	feat(inference): package shared preprocessing and label map	Lokal serving, Colab export bundle içindeki aynı transform sözleşmesini ve version'lı label map'i kullanır. Ayrı preprocessing kopyası ve lokal eğitim yasaklanır.	Training-serving golden tensör eşitliği; label sırası registry artifact'ıyla eşleşir.
049	feat(inference): load immutable registry model versions	Colab'den terfi eden model alias'ı lokal entegrasyonda immutable version'a çözülür; bundle indirilir, checksum ve schema doğrulanır. Pod restart'ında sessiz değişim olmaz.	Yanlış checksum, uyumsuz schema veya ulaşılamayan registry readiness'i düşürür; liveness'i gereksiz bozmaz.
050	feat(inference): implement single and batch prediction service	Framework bağımsız servis sınıfı tekli/batch tensörleme, no-grad, confidence ve model metadata döndürür; batch üst sınırı config'tir.	Sıra korunumu, batch/tekli tutarlılığı, boş/aşırı batch ve deterministik sonuç testleri.
051	test(inference): add golden parity and skew detection	Colab export bundle içindeki sabit IQ, preprocessing hash, logits, sınıf ve confidence değerleri lokal serving sonucuyla karşılaştırılır; bu bir entegrasyon testidir, model testi değildir.	Model/preprocess değişikliği golden testi kasıtlı güncelleme olmadan geçemez.
052	feat(api): create FastAPI app and health endpoints	App factory, dependency injection, /health/live ve model hazır olma durumunu yansıtan /health/ready uçları eklenir.	Model var/yok durumlarında health contract testleri; import sırasında ağır model indirme yapılmaz.
053	feat(api): validate prediction request contracts	Pydantic şemaları IQ uzunluğu, numeric type, NaN/Inf, payload/batch limiti ve schema_version kontrolü uygular.	Geçerli, eksik, bozuk, aşırı ve uyumsuz sürüm payload'larında beklenen 2xx/4xx kodları snapshot ile doğrulanır.
054	feat(api): add prediction endpoints and error model	POST /v1/predict ve /v1/predict/batch; prediction, confidence, model_version, preprocessing_version, request_id ve duration_ms döndürür. Tutarlı hata gövdesi eklenir.	OpenAPI contract ve integration testleri; iç exception/stack trace istemciye sızmaz.
055	feat(api): add structured safe request logging	JSON log, correlation/request ID, süre, endpoint, status ve model version eklenir. Ham IQ, token ve kişisel alanlar loglanmaz; örnekleme/retention config olur.	Log capture testi yasak alanların bulunmadığını ve request_id zincirini doğrular.
056	test(api): add contract integration and latency smoke suite	Uvicorn süreçli gerçek HTTP testleri, eşzamanlı istek, graceful shutdown, model yok, büyük payload ve kısa yük smoke senaryosu eklenir.	Lokal API işi OpenAPI artifact'ı üretir; Colab model bundle'ı ile hata oranı sıfır ve başlangıç p95 bütçesi içinde olmalıdır. v0.3.0-api hazırdır.

Faz 6 - Container, Kubernetes ve gerçek CI/CD (057-064)

Aynı immutable uygulama image'ını staging ve production'a kanıt zinciriyle taşımak.

G6: OCI digest üretilmiş, taranmış, kind/staging smoke geçmiş; production terfisi aynı digest ve manuel onayla yapılabilir.

No	Önerilen commit	Uygulama kapsamı	CI / kabul kanıtı
057	build(container): add minimal non-root image	Pinned digest'li base, multi-stage build, yalnız serve bağımlılıkları, non-root kullanıcı, read-only uyumlu yollar, HEALTHCHECK ve init sinyali yönetimi eklenir.	docker inspect ile user/health; container unit smoke; image içinde test/data/credential bulunmadığı kontrol edilir.
058	build(compose): add local service and tracking stack	Compose profilleri lokal API ve monitoring bileşenlerini kurar; yalnız Colab'den terfi etmiş immutable model version kullanılır. Lokal training servisi bulunmaz.	compose config doğrulama; temiz volume ile ayağa kalkma ve predict smoke testi.
059	ci(container): build scan sbom and smoke test image	Build cache, SBOM, zafiyet taraması, image boyut raporu ve container içinden API smoke işi eklenir. PR image'ı registry'ye push edilmez.	High/critical politika ihlali ve non-root/health smoke başarısızlığı merge'i durdurur; SBOM artifact olarak saklanır.
060	feat(k8s): add base deployment service and config	Kustomize base ile Deployment, Service, ConfigMap ve Secret referansları; immutable image/model parametreleri ve namespace sınırı eklenir.	kustomize build ve schema validation; Secret değerleri repoda yoktur.
061	feat(k8s): add probes resources and rollout safety	Startup/readiness/liveness, request/limit, RollingUpdate, termination grace, securityContext ve gerekirse HPA/PDB için ölçülü varsayılanlar eklenir.	Probe'lar doğru uçlara gider; runAsNonRoot/readOnlyRootFilesystem ve limitler policy testinden geçer.
062	test(k8s): add manifest policy and kind smoke	Kube manifest lint/schema/policy kontrolleri ve ephemeral kind cluster'da deploy-ready-predict-rollout testi CI'a eklenir.	Pod Ready, Service yanıtı ve kontrollü rollout başarılı; başarısız yeni revision otomatik olarak başarılı sayılmaz.
063	ci(release): publish immutable OCI artifacts	Main'de yalnız yeşil commit için image SHA ve SemVer tag ile registry'ye push edilir; digest, SBOM ve build provenance release metadata'sına yazılır.	Tag overwrite yasak; deploy girdisi tag değil digest'tir. OIDC/kısa ömürlü kimlik tercih edilir.
064	cd(deploy): promote digest through staging and production	Staging aynı digest ile otomatik deploy edilir, migration olmayan servis için smoke/sentetik trafik koşar. Production yalnız SemVer + environment onayıyla aynı digest'i alır; önceki digest rollback girdisidir.	Deploy concurrency kilidi, timeout, post-deploy health ve rollback testi. Kod image'ı ile MODEL_VERSION ayrı fakat birlikte release manifestinde kayıtlıdır.

Faz 7 - Gözlemlenebilirlik, drift, retraining ve ürünleşme (065-072)

Sistemin bozulmasını görmek, kanıtsız otomasyonu engellemek ve güvenli model iyileştirme döngüsünü kapatmak.

G7: Trafik-dashboard-alert-drift-candidate-promotion/ret-rollback zinciri uçtan uca gösterilmiş; v1.0.0-mvp yayımlanabilir.

No	Önerilen commit	Uygulama kapsamı	CI / kabul kanıtı
065	feat(monitoring): instrument service metrics	Request/error/in-flight, latency histogramı ve prediction sınıf sayımları /metrics'te sunulur. Endpoint, status ve model_version düşük cardinality label'dır; request_id label değildir.	Metrik isim/label contract testi; örnek trafik sonrası sayaç ve histogram değişimi doğrulanır.
066	ops(prometheus): add scrape rules and actionable alerts	Prometheus config, API/container scrape, availability, error rate, p95 latency, no-ready-replica ve scrape-absent alarm kuralları eklenir.	promtool benzeri rule testi ve sentetik eşik ihlali; her alarm runbook bağlantısı taşır.
067	ops(grafana): provision dashboards and initial SLOs	Trafik, p50/p95/p99, hata, kaynak, model version ve tahmin dağılımı panelleri; availability/latency SLO ve hata bütçesi görünümü provision edilir.	Boş panel veya hatalı sorgu yok; kontrollü yükte panel verileri Prometheus sorgusuyla çapraz kontrol edilir.
068	feat(drift): collect privacy-safe inference summaries	Raw IQ yerine izinli özetler (I/Q mean-std, power, amplitude, phase, varsa SNR, prediction) pencere kimliğiyle saklanır. Retention ve silme politikası config'tir.	Ham payload/log sızıntı testi; pencere sayımı, schema version ve model version eksiksiz olmalı.
069	feat(drift): build versioned Evidently drift pipeline	Referans snapshot ile current pencere karşılaştırılır; minimum örnek, feature/prediction drift eşikleri, ardışık ihlal ve sentetik shift raporu üretilir.	Önerilen başlangıç: en az 1000 örnek, 3 ardışık ihlal, 24 saat cooldown; veri keşfiyle config güncellenir. Normal/shift fixture testleri.
070	feat(retraining): orchestrate audited candidate runs	Drift tetikleyicisi otomatik production eğitimi yapmaz; onaylı Colab retraining runbook'unu ve sabit config'i hazırlar. Colab validate -> model-tests -> train -> evaluate -> registry candidate zinciri manuel başlatılır ve audit kaydı tutulur.	Yetersiz örnek, cooldown, Colab GPU yokluğu, oturum kopması, validation/test/training ve artifact upload hataları güvenli sonlanır; champion değişmez.
071	feat(governance): add approval promotion and rollback workflow	Champion-challenger kapısı kritik recall, macro-F1, latency ve veri uygunluğunu kontrol eder; manuel onay/ret nedeni kaydeder. Model alias ve deployment manifesti birlikte, atomik prosedürle güncellenir.	Kötü challenger ret, iyi challenger onay, başarısız rollout rollback ve audit bütünlüğü uçtan uca test edilir.
072	docs(release): complete runbooks demo and mvp release	README quickstart, data/model card, ADR'ler, operasyon/güvenlik/troubleshooting runbook'ları; örnek trafik, alarm, drift, retraining, terfi/ret ve rollback demo script'i tamamlanır.	Temiz makine provası, tüm CI/CD yeşil, release manifesti kod SHA + image digest + data rev + model version içerir; v1.0.0-mvp yayımlanır.

8. Başlangıç eşikleri ve kalibrasyon

- Model için sabit yüzde 90 kullanılmaz. İlk baseline ve veri dağılımı görüldükten sonra macro-F1, kritik sınıf recall ve SNR dilimi eşikleri config'e alınır.
- Challenger başlangıç politikası: kritik recall tabanlarını karşıla; macro-F1'de anlamlı iyileşme göster veya 0.005 mutlak non-inferiority içinde kalıp ölçülmüş latency/size faydası getir. Değerler ADR ile kesinleşir.
- Drift başlangıç önerisi: en az 1000 örnek, 3 ardışık ihlal, 24 saat cooldown. Trafik hacmi ve yanlış alarm oranına göre kalibre edilir.
- Colab commit 044 eğitim throughput/VRAM bütçesini; lokal API ve staging ölçümleri ise gerçek serving p95 bütçesini üretir. Eğitim ve serving performansları birbirine karıştırılmaz.

Kapsam dışı backlog

- Gerçek zamanlı radar donanımı ve Kafka/Redpanda streaming ingestion
- Tam cloud IaC, çok kümeli Kubernetes, service mesh ve GitOps platform kurulumu
- Feature store, aktif öğrenme arayüzü ve etiketleme operasyonu
- ONNX/TensorRT/GPU serving, canary ve çok bölgesel yüksek erişilebilirlik
- Kurumsal IAM/RBAC/TLS/rate limiting çözümünün tamamı; MVP'de güvenli sınırlar ve entegrasyon noktaları hazırlanır

9. Son demo ve release kanıtı

1. DVC revision ve validation raporunu göster.
2. Temiz Colab GPU runtime'ında model testlerini ve training run'ı başlat; Drive checkpoint'ini, MLflow Git SHA/data rev/GPU/metrik/artifact kaydını göster.
3. Registry candidate'ın kalite kapısından geçtiğini veya neden reddedildiğini göster.
4. Pinli model version ile OCI image'ı staging'e aynı digest üzerinden deploy et.
5. Geçerli ve hatalı IQ istekleri gönder; model_version, request_id ve hata contract'ını doğrula.
6. Grafana'da trafik/latency/error/model panellerini ve sentetik alarmı göster.
7. Kaydırılmış veriyle drift raporu ve cooldown/min-sample davranışını göster.
8. Challenger promotion/ret ve önceki image/model rollback senaryosunu çalıştır.
9. Release manifestinde Git SHA + image digest + DVC rev + MLflow model version zincirini göster.

Tamamlanma tanımı

Bu yol haritası süre sözü vermez. İlerleme yalnız ilgili commit ve çıkış kapısı CI/CD kanıtıyla tamamlandığında sayılır.